Q. Write code that display message whenever user i/p any characters or Press any key of DPAD (directed PAD)?

```
import android.os.Bundle;
import android.view.KeyEvent;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity
{
    private EditText userInput;
    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        userInput = findViewById(R.id.userInput);
        // Setup a key listener for editText
        userInput.setOnKeyListener(new View.OnKeyListener(){
            @Override
            public boolean onKey(View view, int keyCode, KeyEvent keyevent)
            {
                if(keyevent.getAction() == KeyEvent.ACTION_DOWN)
                {
                    showMessage("Key pressed :" + keyevent.getDisplayLabel());
                    return true;
                }
                return false;
            });
        }
        private void showMessage(String msg)
        {
```

.Toast.makeText(this, msg, Toast.LENGTH_SHORT
).show();
}

xml file:

```
<EditText
    android:id="@+id/editText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:hint="Type something"
/>
```

// for check specific keys e.g.
'D', 'O', 'A', 'D'

```
if( keyCode == KeyEvent.KeyCode_D ||
    keyCode == KeyEvent.Kg.Code_A ||
    )
```

Practical No:

Q. Write an application that suggest next text on cities names whenever user type some city in the i/p box and on selection of any option Autocomplete that city name in to the box?

```
import android.os.Bundle;
import android.text.Editable;
import android.text.TextWatcher;
import android.widget.ArrayAdapter;
import android.widget.AutoCompleteTextView;
public class MainActivity extends
AppCompatActivity
{
    private AutoComplete cityInput;
    private ArrayAdapter<String> adapter;
@Override
    protected void onCreate (Bundle
savedInstanceState)
    {
        super.onCreate(savedInstanceState);
```

```
setContentView(R.layout.activity_main);

// Simulated City data
String[] cities = {"New York", "Los Angeles",
"London", "Paris", "Tokyo", "Sydney"};
ArrayAdapter adapter = new ArrayAdapter<>(this,
android.R.layout.simple_dropdown_item_1line, cities);
cityInput.setAdapter(adapter);

// Adding text change listener for
dynamic suggestions

cityInput.addTextChangedListener(new
TextWatcher() {
    @Override
    public void beforeTextChanged(CharSequence
cs, int start, int before, int count)
    {
    }
    @Override
    public void onTextChanged(CharSequence cs,
int start, int before, int count)
    { }
    @Override
    public void afterTextChanged(Editable
editable)
    {
        // filter the suggestions based on
user input
        adapter.getFilter().filter(editable.
toString());
    }
});
}
}
```

Xml file:

```
<AutoCompleteTextView
    android:id="@+id/cityInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Type a city name"
/>
```

2. Write a note on the failure of window phone and success of android / iOS?

→ The rise of Android and the relative decline of Windows Phone in the mobile OS market can be attributed to various factors.

Windows Phone Failure:-

1) Late Entry:

- Windows Phone entered the smartPhone market relatively late, in 2010, well after iOS and Android had already established themselves as dominant players.
- By that time, the app ecosystems of iOS and Android were flourishing, making it difficult for Windows Phone to "compete" in terms of available apps (~~as a third platform~~) and developers support.

2) Lack of Developers Support:

⇒ Windows Phone struggled to attract a robust developer community.

⇒ Developers were already investing heavily in iOS and Android, and building apps for a third platform with a smaller user base was less attractive

3) Limited hardware choices:

⇒ Windows phone faced limitations in terms of hardware diversity. Android, in contrast, was available on a wide range of devices from various manufacturers, catering to various price points and user preferences. This gave Android a significant advantage in terms of market share.

4: App Gap:

⇒ The app gap was a critical issue for windows Phone.

⇒ Many popular apps and services were either unavailable or had inferior versions compared to their iOS and Android counterparts.

⇒ This discouraged users from adopting the platform

5) Corporate Strategy:

- Microsoft's corporate strategy also shifted over time.
- They transitioned from windows Phone to Windows 10 mobile and later windows Phone was effectively abandoned in favour of other platforms like iOS and Android.
- This left existing windows Phone users in limbo and discouraged new users from adopting the platform.

Android OS Success:

1: Open Source and Customization:

- Android's open-source nature allowed manufacturers to customize and adapt the OS to suit their hardware.
- This flexibility led to a wide variety of Android devices, appealing to a broad range of consumers.

2) App Ecosystem:

⇒ Android's Google Play store grew rapidly offering a vast array of apps and games.

⇒ This rich ecosystem attracted both developers and users, making Android a compelling platform for app development.

3. Market share:

⇒ Android quickly gained a dominant market share due to its availability on a wide range of devices.

⇒ This attracted developers looking to reach the largest user base, creating a positive feedback loop.

4. Google integration:

⇒ The deep integration of Google services, such as Gmail, maps, and Search, made Android a seamless choice for users already invested in the Google ecosystem.

5) Constant improvement:

⇒ Google has consistency updated and improved Android with new features and enhancements, keeping it competitive and appealing to both users and manufacturers.

1. Discuss Components of a screen and adapting to display orientation?

Definition:

"Screen Components typically refers to the various UI elements or widgets that make up the user interface of the Android app."

⇒ These components are responsible for presenting information, accepting user input, and facilitating interactions.

Components:-

- Layouts
- Views
- Widgets
- Intents
- Fragments
- Styles and Themes
- Activity
- Resources
- Listeners & Event handling
- Resource files

Containers:

⇒ Hold and manage multiple views or fragments. Containers group views or fragments together, simplifying the organization and management of complex user interfaces and enhancing modularity.

Resource Files:

⇒ Define layouts, drawables, values, and more.

⇒ Resource file store assets, configuration data, and design elements, allowing for easy modification and centralization of resources used throughout the application.

Resources:

⇒ Store text, colors, dimensions, and themes.

⇒ Resources encompass data and assets that application use to define visual elements, text content, and styles, ensuring flexibility and consistency in design.

Listeners and Event Handling:-

⇒ Listeners are objects or methods that "listen" for specific events or actions, such as button clicks, text input, touch gestures, and more. When the specified event occurs, the listener triggers a callback or method to handle that event.

⇒ Event handling involves responding to user interactions with the app's UI elements. These interactions include clicking buttons, typing in text fields, swiping, tapping and more.

⇒ Event handling allows to define what should happen when these interactions occur.

Adapting to display Orientation in app:

⇒ When the screen orientation of an android device changes, the current activity being displayed is destroyed

and recreated automatically to redraw its content in the new orientation.

⇒ The design of layout should be in such a way that they can adapt to different screen orientations.

⇒ For that the following things need to do:

(1) Use relative dimensions:

- Use relative dimensions instead of absolute dimensions.
- This will ensure that your views are scaled correctly when the screen orientation changes.
- Relative dimensions means that specifying the dimensions of a view relative to its parent's view.

(2) Use responsive layouts:

- Use layouts that are designed to be responsive to different screen sizes and orientations.

ConstraintLayout is a good choice for designing responsive layouts or RelativeLayout.

- Use drawables:

⇒ Use drawables and colors that are compatible with different screen orientations.

⇒ for example: Avoid using hard-coded colors in layouts.

- Instead, use android resource system to define color.

- Use onConfigurationChanged():

⇒ Use the onConfigurationChanged() callback method to handle orientation changes explicitly.

⇒ In this method you can resize and responsive views as needed.

Note:

⇒ android: configChanges attributes can be use in activity manifest file to specify which configuration changes activity and can handle without being destroyed.

for e.g:

- To specify in activity handle orientation changes without being destroyed.
- This will prevent activity from being destroyed and recreated when the screen orientation changes, which can improve performance.

Q. Write the code to take i/p from user along with validation that user can enter only number to not a text, calculate its power and display in another un-editable text field.

xml file:-

```
<LinearLayout
    .....
<EditText
    android:id="@+id/baseEditText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter base number:"
/>
<EditText
    android:id="@+id/exponentEditText"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter exponent"
/>
<Button
    android:id="@+id/calculateButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Calulate" />

<TextView
    android:id="@+id/result"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="" />
</LinearLayout>
```

Java file:

```
import android.os.Bundle;
import android.text.InputFilter;
import android.text.InputType;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import android.appcompat.app.AppCompatActivity;
```

```
Public class MainActivity extends AppCompatActivity
{
    Private EditText baseET;
    Private EditText expET;
    Private Button btn;
    Private TextView result;

    @Override
    Protected void onCreate(Bundle savedInstanceState)
    {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        baseET = findViewById(R.id.baseEditText);
        expET = findViewById(R.id.exponentEditText);
        btn = findViewById(R.id.calculateButton);
        result = findViewById(R.id.result);
    // Set the edittext to accept only
    numbers
        baseET.setInputType(InputType.TYPE_CLASS_NUMBER);
        expET.setInputType(InputType.TYPE_CLASS_NUMBER);

    // Disable the O/P textview
        result.setEnabled(false);

        btn.setOnClickListener(new View.OnClickListener
        ()
        {
            @Override
            Public void onClick(View v)
            {
                calculatePower();
            }
        });
    }
    Private void calculatePower()
    {
        String baseSts = baseET.getText().toString();
        String expSts = expET.getText().toString();

        if(!baseSts.isEmpty() && !expSts.isEmpty())
        {
            try{
                double base = Double.parseDouble(baseSts);
                double exponent = Double.parseDouble(expSts);
                double result = Math.pow(base, exponent);
```

```
            result.setText("Result:" + result);
        }
        catch (NumberFormatException e)
        {
            result.setText("Invalid input. Please enter
            valid number");
        }
    }
}
else
{
    result.setText("Please enter both base &
    exponent");
}
}
```